SQL INTERVIEW PREPARATION

Practical study notes with examples

A daily-read reference for SQL fundamentals, joins, advanced queries, ETL validation, indexing, normalization and BigQuery.

Learning approach

Read one section at a time. Run the examples where possible, understand what happens to the rows, and practise explaining the result aloud. The examples use broadly familiar SQL; BigQuery-specific notes are called out where relevant.

How to use these notes

These notes are designed as a reference handbook rather than a fixed study schedule. You can revise a small section whenever you have 20–30 minutes.

- First understand the input rows and the expected output.
- Then read the query and explain each clause in your own words.
- For interview preparation, focus on why a query works, not only on remembering syntax.

Example tables used throughout

Employees	employee_id	name	department	salary
	101	Amit	IT	80000
	102	Neha	HR	60000
	103	Ravi	IT	90000
	104	Sara	Finance	70000

1. SQL fundamentals

SELECT, FROM and WHERE

A SELECT query reads rows from a table, optionally filters them, and returns the requested columns. Think of FROM as the data source, WHERE as the row filter, and SELECT as the columns you want to see.

```
SELECT employee_id, name, salary
FROM Employees
WHERE department = 'IT';
```

This returns only employees in IT, showing their ID, name and salary. The WHERE condition is applied to rows before the final result is returned.

DISTINCT

DISTINCT removes duplicate result rows based on all selected columns.

```
SELECT DISTINCT department
FROM Employees;
```

If several employees belong to IT, IT appears only once in this result.

ORDER BY and LIMIT

```
SELECT name, salary
FROM Employees
ORDER BY salary DESC
LIMIT 2;
```

ORDER BY sorts the result. DESC means highest to lowest; ASC means lowest to highest. LIMIT restricts the number of returned rows. Syntax varies by database; BigQuery supports LIMIT.

GROUP BY and aggregate functions

GROUP BY collects rows into groups. Aggregate functions calculate one value for each group.

```
SELECT department,
       COUNT(*) AS employee_count,
       AVG(salary) AS average_salary,
       MAX(salary) AS highest_salary
FROM Employees
GROUP BY department;
```

The database groups employees by department and calculates the count, average and maximum salary for each department.

Common aggregates: COUNT(), SUM(), AVG(), MIN(), MAX(). COUNT(*) counts rows; COUNT(column) ignores NULL values in that column.

WHERE versus HAVING

```
SELECT department, AVG(salary) AS avg_salary
FROM Employees
WHERE salary >= 60000
GROUP BY department
HAVING AVG(salary) > 70000;
```

WHERE filters individual rows before grouping. HAVING filters groups after aggregate values have been calculated.

CASE WHEN

```
SELECT name, salary,
       CASE
         WHEN salary >= 80000 THEN 'High'
         WHEN salary >= 65000 THEN 'Medium'
         ELSE 'Low'
       END AS salary_band
FROM Employees;
```

CASE creates conditional output, similar to an if/else decision.

NULL, IS NULL and COALESCE

NULL means a value is missing or unknown. It is not the same as zero or an empty string. Use IS NULL or IS NOT NULL to test it.

```
SELECT *
FROM Employees
WHERE department IS NOT NULL;

SELECT name, COALESCE(department, 'Unknown') AS department
FROM Employees;
```

COALESCE returns the first non-NULL expression. In BigQuery, IFNULL is also available for a two-argument replacement.

Common filter operators

```
WHERE salary BETWEEN 60000 AND 80000
WHERE department IN ('IT', 'HR')
WHERE name LIKE 'A%'
WHERE salary > 60000 AND department = 'IT'
```

BETWEEN includes both endpoints. IN checks membership in a list. LIKE supports patterns: % means any number of characters and _ means one character.

2. Joins and subqueries

A JOIN combines rows from two tables using a related column. Before choosing a join, ask: which rows must be retained, and can one key match multiple rows on the other side?

INNER JOIN

```
SELECT e.name, d.department_name
FROM Employees e
INNER JOIN Departments d
  ON e.department = d.department_code;
```

Returns only rows that have a match in both tables.

LEFT JOIN

```
SELECT e.name, d.department_name
FROM Employees e
LEFT JOIN Departments d
  ON e.department = d.department_code;
```

Returns every row from the left table. If no department matches, the department columns are NULL. This is useful for finding source records without a matching lookup or target record.

RIGHT JOIN, FULL OUTER JOIN and CROSS JOIN

- **RIGHT JOIN:** retains every row from the right table and matching rows from the left.
- **FULL OUTER JOIN:** retains all rows from both sides; unmatched columns are NULL.
- **CROSS JOIN:** produces every possible pair of rows. If one table has 3 rows and the other has 4, the result has 12 rows.

SELF JOIN and multiple joins

```
SELECT e.name AS employee,
       m.name AS manager
FROM Employees e
LEFT JOIN Employees m
  ON e.manager_id = m.employee_id;
```

A self join joins a table to itself, for example to connect employees to their managers. You can chain multiple joins to bring together facts and lookup descriptions.

Subqueries and EXISTS

```
SELECT name
FROM Employees
WHERE salary > (
  SELECT AVG(salary) FROM Employees
);
```

A subquery is a query nested inside another query. The inner query calculates the average salary; the outer query returns employees earning more than that value.

```
SELECT c.customer_id
FROM Customers c
WHERE EXISTS (
  SELECT 1
  FROM Transactions t
  WHERE t.customer_id = c.customer_id
);
```

EXISTS checks whether the subquery returns at least one row. It is often useful for existence checks and avoids needing to return the matching values themselves. Be careful with NOT IN if the subquery can return NULL; NOT EXISTS is often safer.

CTE versus subquery

```
WITH high_paid AS (
  SELECT *
  FROM Employees
  WHERE salary > 75000
)
SELECT name, salary
FROM high_paid;
```

A Common Table Expression (CTE) gives a named result set for a query. It can make multi-step transformations easier to read. A subquery embeds the logic directly. A CTE is not automatically a performance improvement; optimizer behaviour depends on the database.

3. Advanced SQL

ROW_NUMBER, RANK and DENSE_RANK

```
SELECT name, department, salary,
       ROW_NUMBER() OVER (
         PARTITION BY department
         ORDER BY salary DESC
       ) AS row_num,
       RANK() OVER (
         PARTITION BY department
         ORDER BY salary DESC
       ) AS salary_rank,
       DENSE_RANK() OVER (
         PARTITION BY department
         ORDER BY salary DESC
       ) AS dense_salary_rank
FROM Employees;
```

All three assign ordering values within each department. ROW_NUMBER always gives a unique sequence (though ties need a deterministic tie-breaker). RANK leaves gaps after ties; DENSE_RANK does not.

- **ROW_NUMBER:** 1, 2, 3, 4
- **RANK:** 1, 1, 3, 4
- **DENSE_RANK:** 1, 1, 2, 3

LAG and LEAD

```
SELECT customer_id, transaction_date, amount,
       LAG(amount) OVER (
         PARTITION BY customer_id
         ORDER BY transaction_date
       ) AS previous_amount,
       LEAD(amount) OVER (
         PARTITION BY customer_id
         ORDER BY transaction_date
       ) AS next_amount
FROM Transactions;
```

LAG accesses a value from a previous row; LEAD accesses a value from a following row within the ordered window. Useful for comparing current and previous transactions or detecting changes.

FIRST_VALUE and LAST_VALUE

```
SELECT customer_id, transaction_date, amount,
       FIRST_VALUE(amount) OVER (
         PARTITION BY customer_id
         ORDER BY transaction_date
       ) AS first_amount
FROM Transactions;
```

These return the first or last value in a window frame. LAST_VALUE can surprise people because the default frame may end at the current row. Specify an explicit frame when you need the final value across the full partition.

PARTITION BY versus GROUP BY

GROUP BY collapses many input rows into one output row per group. PARTITION BY in a window function keeps the individual rows and calculates values across a related group of rows.

```
-- One row per department
SELECT department, AVG(salary)
FROM Employees
GROUP BY department;

-- Every employee row remains
SELECT name, department, salary,
       AVG(salary) OVER (PARTITION BY department) AS dept_avg
FROM Employees;
```

Running total and moving average

```
SELECT account_id, txn_date, amount,
       SUM(amount) OVER (
         PARTITION BY account_id
         ORDER BY txn_date
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS running_total
FROM Transactions;
```

The window frame starts at the first row in each account and ends at the current row. A moving average can use a frame such as ROWS BETWEEN 2 PRECEDING AND CURRENT ROW.

Top N rows per group

```
WITH ranked AS (
  SELECT *,
         ROW_NUMBER() OVER (
           PARTITION BY department
           ORDER BY salary DESC, employee_id
         ) AS rn
  FROM Employees
)
SELECT *
FROM ranked
WHERE rn <= 2;
```

This returns the top two employees per department. The employee_id tie-breaker makes the ordering deterministic when salaries are equal.

Set operators

- **UNION:** combines results and removes duplicate rows.
- **UNION ALL:** combines results and keeps duplicates; usually less work than UNION.
- **INTERSECT:** returns rows common to both queries.
- **EXCEPT:** returns rows from the first query that are absent from the second. BigQuery Standard SQL uses EXCEPT DISTINCT and INTERSECT DISTINCT.

INSERT, UPDATE, DELETE and MERGE

```
INSERT INTO Employees (employee_id, name, department, salary)
VALUES (105, 'Maya', 'IT', 72000);

UPDATE Employees
SET salary = 75000
WHERE employee_id = 105;

DELETE FROM Employees
WHERE employee_id = 105;
```

MERGE is especially useful in ETL pipelines because it can update matched target records and insert new records in one statement.

```
MERGE Target T
USING Source S
ON T.customer_id = S.customer_id
WHEN MATCHED THEN
  UPDATE SET T.name = S.name
WHEN NOT MATCHED THEN
  INSERT (customer_id, name)
  VALUES (S.customer_id, S.name);
```

Exact MERGE syntax and supported clauses vary by database. In BigQuery, MERGE is commonly used for idempotent incremental loads.

4. Practical SQL problems for ETL and data engineering

Find duplicate business keys

```
SELECT customer_id, COUNT(*) AS row_count
FROM Source
GROUP BY customer_id
HAVING COUNT(*) > 1;
```

This identifies keys that occur more than once. It does not decide which row should be retained.

Remove duplicates and keep the latest record

```
SELECT *
FROM Source
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY customer_id
  ORDER BY updated_at DESC, ingestion_id DESC
) = 1;
```

The rows are grouped logically by customer_id and ranked newest first. QUALIFY filters after the window function in BigQuery. In databases without QUALIFY, put the ranking in a CTE and filter it in the outer query.

Find the second-highest distinct salary

```
SELECT MAX(salary) AS second_highest_salary
FROM Employees
WHERE salary < (
  SELECT MAX(salary) FROM Employees
);
```

The inner query finds the highest salary. The outer query finds the maximum salary below it. This returns the second-highest distinct value.

Find source records missing in target

```
SELECT S.*
FROM Source S
LEFT JOIN Target T
  ON S.business_key = T.business_key
WHERE T.business_key IS NULL;
```

This is an anti-join pattern. It is useful for identifying records that have not reached the target.

Find customers with no transactions

```
SELECT C.customer_id
FROM Customers C
LEFT JOIN Transactions T
  ON C.customer_id = T.customer_id
WHERE T.customer_id IS NULL;
```

Compare source and target values

```
SELECT S.customer_id,
       S.status AS source_status,
       T.status AS target_status
FROM Source S
JOIN Target T
  ON S.customer_id = T.customer_id
WHERE S.status IS DISTINCT FROM T.status;
```

IS DISTINCT FROM compares values while treating NULL safely. BigQuery supports it. In databases without this operator, use explicit NULL-aware comparison logic.

Incremental load using a watermark

```
SELECT *
FROM Source
WHERE updated_at > @last_successful_watermark;
```

A watermark records how far a successful load has processed. In production, avoid relying only on MAX(updated_at) from the target: late-arriving rows and timestamp ties can be missed. A safer design uses a persisted control-table watermark, a small overlap/reprocessing window, and an idempotent MERGE into the target.

Reconciliation

```
SELECT COUNT(*) AS row_count,
       SUM(amount) AS total_amount,
       MIN(transaction_date) AS min_date,
       MAX(transaction_date) AS max_date
FROM Source;
```

Compare source and target row counts, sums, date ranges, key counts and business-level aggregates. Matching row counts alone do not prove that the data is correct.

5. SQL indexing and query performance

What is an index?

An index is an additional data structure maintained by a database to help locate rows without scanning every row in a table. A common index structure in traditional relational databases is a B-tree or B+ tree.

Imagine a book: instead of reading every page to find a topic, you use the index to jump to the relevant page. A database index provides a similar shortcut for qualifying searches.

```
-- Example for a traditional relational database
CREATE INDEX idx_employee_department
ON Employees(department);
```

A query filtering on department may be able to use this index to find matching row locations more efficiently.

How does an index improve performance?

- Without a useful index, the engine may scan many or all rows to find matching records.
- With a useful index, the engine can navigate the index to locate matching keys and then fetch the required rows.
- Indexes can help filters, joins, ordering and uniqueness checks, depending on the database and query plan.

An index does not guarantee a faster query. If a query returns a large proportion of the table, a full scan may be cheaper than using the index.

Common index types

- **Clustered index:** in systems that support this concept, controls or strongly influences how table rows are organized by the index key. A table generally has one clustered organization.
- **Non-clustered index:** a separate structure containing indexed keys and row locators or references.
- **Unique index:** enforces uniqueness of the indexed key or key combination.
- **Composite index:** indexes multiple columns, for example (department, salary). Column order matters.
- **Covering index:** contains the columns needed by a query, potentially avoiding extra table lookups. Implementation varies by database.

Index seek, index scan and table scan

- **Index seek:** navigates to a specific key range; often efficient for selective filters.
- **Index scan:** reads some or all of an index's entries.
- **Table/full scan:** reads table data to evaluate rows. This can be appropriate when many rows are needed.

Use the database's execution plan to see what the optimizer actually chose. Do not assume an index is being used just because it exists.

Index trade-offs

- Indexes consume storage.
- INSERT, UPDATE and DELETE operations may become slower because indexes must also be maintained.
- Too many or poorly chosen indexes can hurt write performance and increase maintenance.
- Choose indexes based on actual workload patterns, selectivity, joins and execution plans.

Important: BigQuery does not use traditional user-created B-tree indexes

BigQuery is a columnar, distributed analytical warehouse. You generally do not create ordinary B-tree indexes with CREATE INDEX as you would in a traditional OLTP database. Instead, performance tuning commonly focuses on:

- **Partitioning:** divides a table into partitions, often by date or timestamp, so queries can skip irrelevant partitions when filters match the partitioning column.
- **Clustering:** organizes data by selected columns within storage blocks, helping BigQuery skip blocks that cannot match filters.
- **Column selection:** avoid SELECT * when only a few columns are needed, reducing bytes read.
- **Materialized views:** can precompute and maintain results for supported query patterns.
- **Query plan and bytes scanned:** inspect execution details, shuffle, stages and data processed.

BigQuery also supports search indexes for certain search workloads. They are a specialized feature, not a direct replacement for ordinary OLTP B-tree indexing.

Interview answer: Traditional databases often use indexes to locate individual rows quickly. BigQuery is columnar and distributed, so I usually optimize using partition pruning, clustering, selecting only required columns, materialized views and execution-plan analysis.

6. SQL normalization

Normalization is the process of structuring relational tables to reduce unnecessary duplication and avoid insert, update and delete anomalies. It separates facts into related tables and connects them using keys.

Why normalize?

- Reduce repeated storage of the same fact.
- Avoid inconsistent updates when the same value appears in many rows.
- Make relationships and data ownership clearer.

Example of a poorly structured table:

```
OrderID | CustomerName | CustomerPhone | Product
1001    | Asha         | 99990000      | Laptop
1002    | Asha         | 99990000      | Mouse
1003    | Ravi         | 88880000      | Keyboard
```

Asha's phone number is repeated. If it changes, several rows must be updated. A customer table separates customer details from order details.

```
Customers
CustomerID | CustomerName | CustomerPhone
1          | Asha         | 99990000
2          | Ravi         | 88880000

Orders
OrderID | CustomerID | Product
1001    | 1          | Laptop
1002    | 1          | Mouse
1003    | 2          | Keyboard
```

First Normal Form (1NF)

A table is in 1NF when each cell contains a single atomic value for the intended attribute and there are no repeating groups or multi-valued columns.

Not ideal: one column contains 'Laptop, Mouse, Keyboard'. **Better:** store each product as a separate row in an order-line table.

Second Normal Form (2NF)

A table is in 2NF when it is in 1NF and every non-key attribute depends on the whole of every candidate key, not just part of a composite key.

Example: in OrderProduct(OrderID, ProductID, ProductName, Quantity), if the key is (OrderID, ProductID) but ProductName depends only on ProductID, that is a partial dependency. Move ProductName to a Product table.

Third Normal Form (3NF)

A table is in 3NF when it is in 2NF and non-key attributes do not depend transitively on a candidate key.

Example: EmployeeID determines DepartmentID, and DepartmentID determines DepartmentName. Store department details in a Department table rather than repeating DepartmentName in every employee row.

Boyce–Codd Normal Form (BCNF)

BCNF is a stronger form of 3NF. For every non-trivial functional dependency $X \rightarrow Y$, X must be a superkey. It addresses certain dependency cases that 3NF can allow.

Normalization versus denormalization

Normalization reduces duplication and supports reliable transactional updates. Denormalization intentionally combines or duplicates data to make analytical queries simpler or faster. Data warehouses often use dimensional models with fact and dimension tables, and may include denormalized structures for reporting.

Remember: normalization is about relational design and dependencies between attributes. It is not the same as concatenating values or formatting strings.

7. Views and STRING_AGG

What is a view?

A view is a named SQL query that behaves like a virtual table. A standard view generally stores the query definition, not a separate copy of the result data. When queried, the database evaluates the view definition against its underlying tables.

```
CREATE VIEW vw_it_employees AS
SELECT employee_id, name, salary
FROM Employees
WHERE department = 'IT';

SELECT * FROM vw_it_employees;
```

Views can simplify complex SQL, expose a controlled subset of columns or rows, and provide a reusable interface to source data.

Standard view versus materialized view

- **Standard view:** stores the query definition; results are normally computed when queried.
- **Materialized view:** stores precomputed results and may be refreshed or incrementally maintained according to database rules. It can improve repeated analytical queries but has eligibility and maintenance considerations.

What does STRING_AGG do?

STRING_AGG combines values from multiple rows into one delimited string. It is an aggregation function. It does not split a string into rows and it is not relational normalization.

```
SELECT customer_id,
       STRING_AGG(product_name, ', ' ORDER BY product_name) AS products
FROM OrderProducts
GROUP BY customer_id;
```

If customer 10 has Laptop, Mouse and Keyboard in separate rows, the output can be one row with a value such as 'Keyboard, Laptop, Mouse'.

Create a view using STRING_AGG in BigQuery

```
CREATE OR REPLACE VIEW dataset.vw_customer_products AS
SELECT
  customer_id,
  STRING_AGG(
    DISTINCT product_name,
    ', ' ORDER BY product_name
  ) AS product_list
FROM dataset.OrderProducts
GROUP BY customer_id;
```

The view presents one row per customer, with the product names combined into a single string. BigQuery supports DISTINCT and ORDER BY inside STRING_AGG.

Can STRING_AGG normalize source data?

This depends on what you mean by normalize. If your goal is to consolidate multiple source rows into one delimited field for downstream display or consumption, STRING_AGG can do that. But it is not database normalization in the 1NF/2NF/3NF sense. In fact, combining multiple values into one string can make the output less atomic and harder to filter or join.

For ETL processing, keep values in separate rows when downstream transformations need to filter, join, validate or aggregate them independently. Use STRING_AGG mainly for presentation, summaries or a defined downstream interface.

Split a delimited string in BigQuery

```
SELECT
  customer_id,
  product
FROM dataset.CustomerProductView,
UNNEST(SPLIT(product_list, ', ')) AS product;
```

SPLIT converts a delimited string into an array; UNNEST turns array elements into rows. This can reverse a simple concatenation, but it is safer to preserve the original normalized rows where possible.

8. Common interview questions and model answers

1. What happens internally when you perform a JOIN?

The database optimizer chooses a physical join strategy based on table sizes, available statistics, ordering, indexes or distribution. Common strategies include nested-loop joins, hash joins and merge joins. The SQL syntax describes the logical result; the execution plan shows the chosen physical approach.

2. Why can a LEFT JOIN increase the number of rows?

If one left-side key matches multiple right-side rows, the left row appears once for every match. A LEFT JOIN preserves unmatched left rows, but it does not guarantee one output row per left key. Check the uniqueness of the join key on the right side.

3. WHERE versus HAVING?

WHERE filters rows before aggregation. HAVING filters groups after GROUP BY and aggregate calculations.

4. How do you remove duplicates and retain the latest row?

Use ROW_NUMBER partitioned by the business key and ordered by the latest timestamp, with a stable tie-breaker. Keep row number 1.

5. Window function versus GROUP BY?

GROUP BY reduces rows to one row per group. A window function calculates across related rows while retaining the original row-level detail.

6. Why does NULL = NULL not return TRUE?

NULL represents an unknown or missing value. Comparing unknown to unknown produces UNKNOWN, not TRUE. Use IS NULL or a NULL-safe comparison such as IS DISTINCT FROM where supported.

7. How would you optimize a slow BigQuery query?

- Check bytes processed and the execution details.
- Filter on partition columns so irrelevant partitions can be pruned.
- Use clustering columns appropriately to reduce scanned blocks.
- Select only the columns required; avoid unnecessary SELECT *.
- Reduce data before expensive joins and avoid accidental many-to-many joins.
- Consider materialized views or pre-aggregation for repeated supported workloads.
- Review shuffle, skew, stages and join strategy in the query plan.

9. Quick revision sheet

Concept	Remember
WHERE	Filters rows before grouping.
HAVING	Filters groups after aggregation.
INNER JOIN	Only matching rows from both sides.
LEFT JOIN	All left rows, plus matching right rows.
ROW_NUMBER	Unique sequence within each window partition.
RANK	Ties share rank; gaps follow.
DENSE_RANK	Ties share rank; no gaps.
GROUP BY	Collapses rows into groups.
PARTITION BY	Defines window groups while retaining rows.
Index	Can speed selective row lookups; adds storage/write cost.
BigQuery tuning	Partitioning, clustering, column pruning, query plan.
1NF	Atomic values; no repeating groups.
2NF	No partial dependency on part of a candidate key.
3NF	No transitive dependency of non-key attributes on a key.
STRING_AGG	Combines values from rows into one delimited string.
View	Reusable named query; standard view usually stores definition.

10. Check your understanding

Try answering these without looking at the notes. If you can explain the reasoning and predict the output, you are building interview-ready understanding.

- What is the difference between INNER JOIN and LEFT JOIN?
- Why can a LEFT JOIN return more rows than the left table?
- When should you use WHERE, and when should you use HAVING?
- How would you keep only the latest record for each business key?
- What is the difference between ROW_NUMBER, RANK and DENSE_RANK?
- How is a traditional database index different from BigQuery partitioning and clustering?
- What problem do 1NF, 2NF and 3NF try to solve?
- Why is STRING_AGG not the same thing as relational normalization?

Suggested learning checkpoint: make sure you can explain these five contrasts clearly: INNER vs LEFT JOIN; WHERE vs HAVING; ROW_NUMBER vs RANK vs DENSE_RANK; indexing vs BigQuery partitioning/clustering; normalization vs STRING_AGG.